

Web Development

>> AJAX

Asynchronous JavaScript and XML

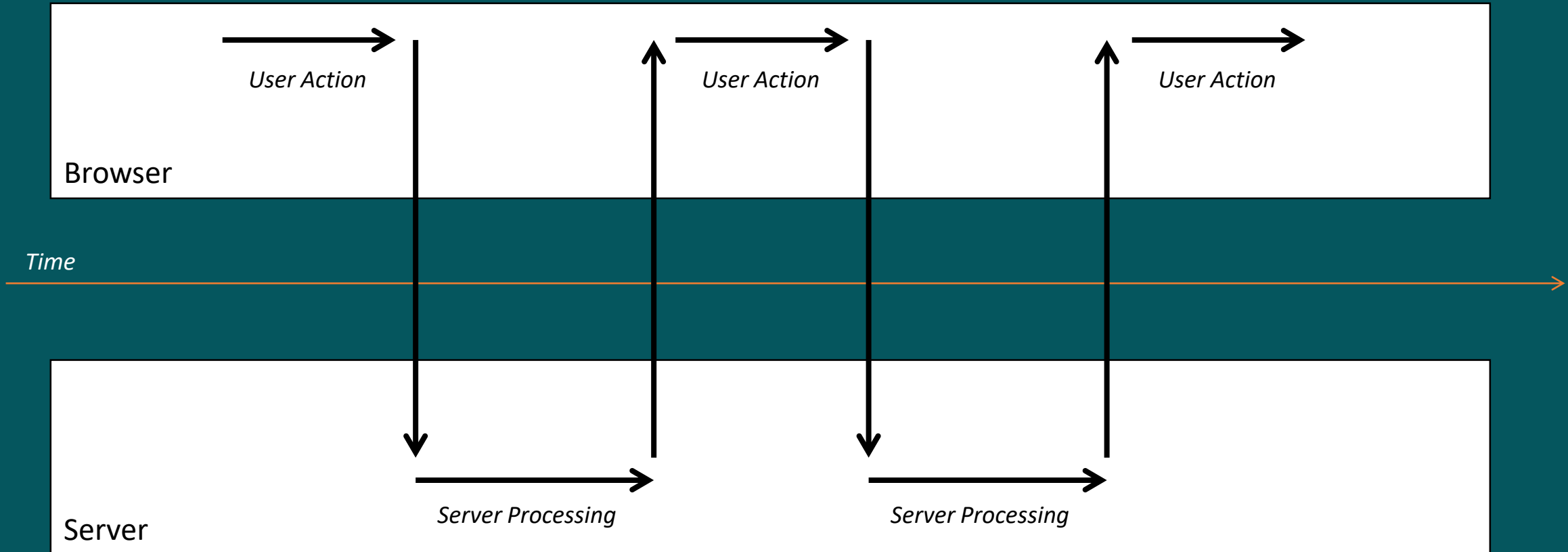
A synchronous

J

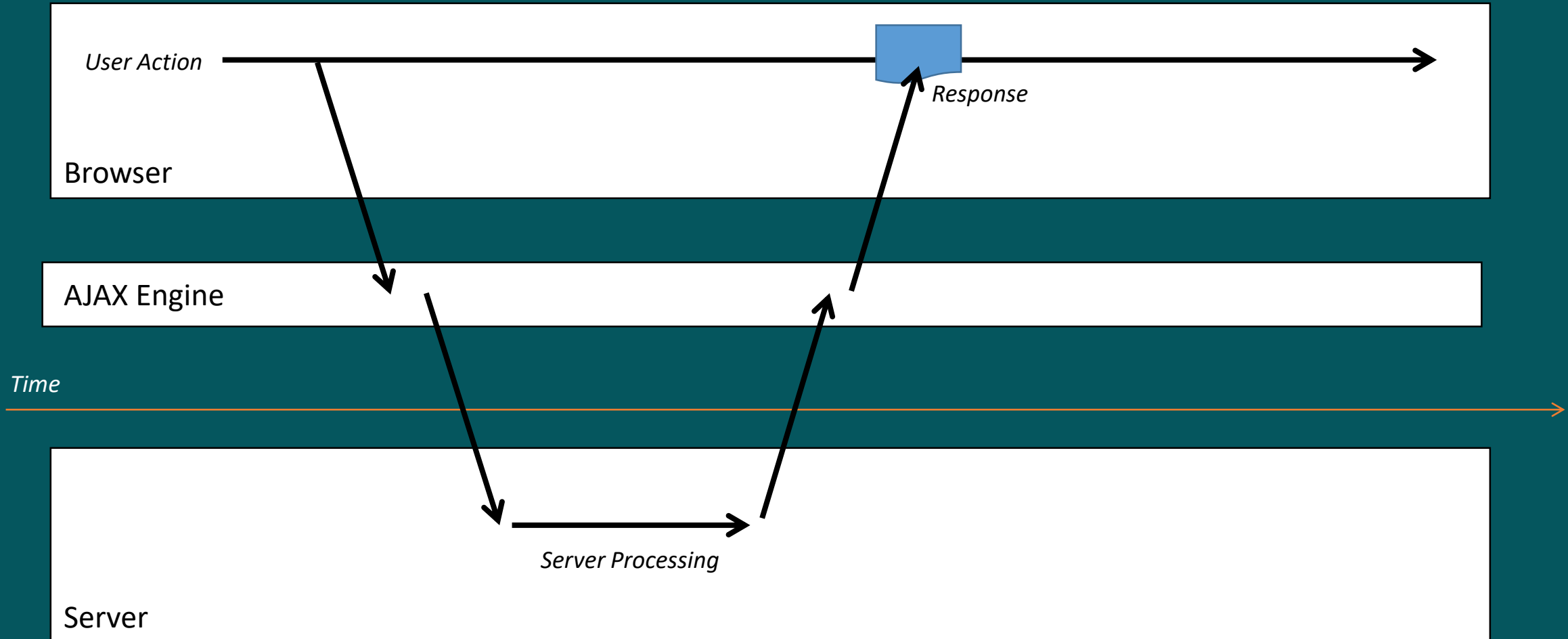
A

X

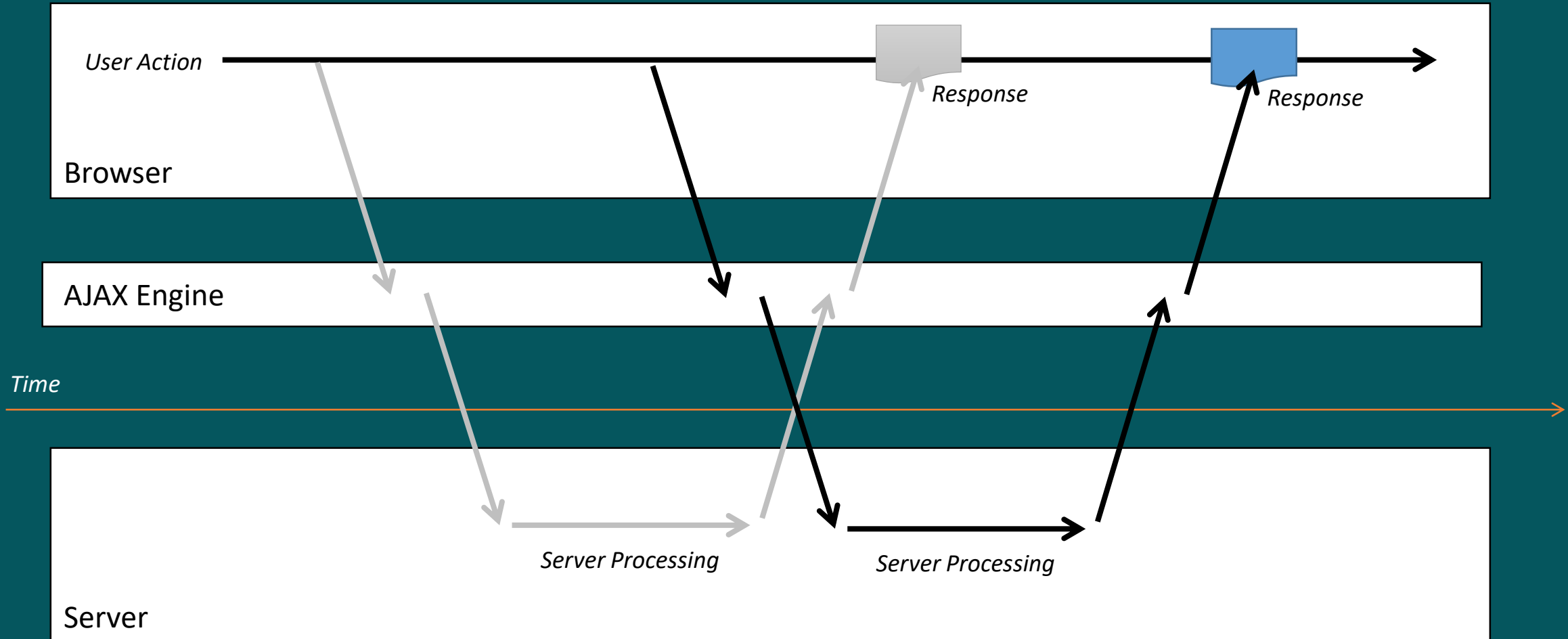
Synchronous Call



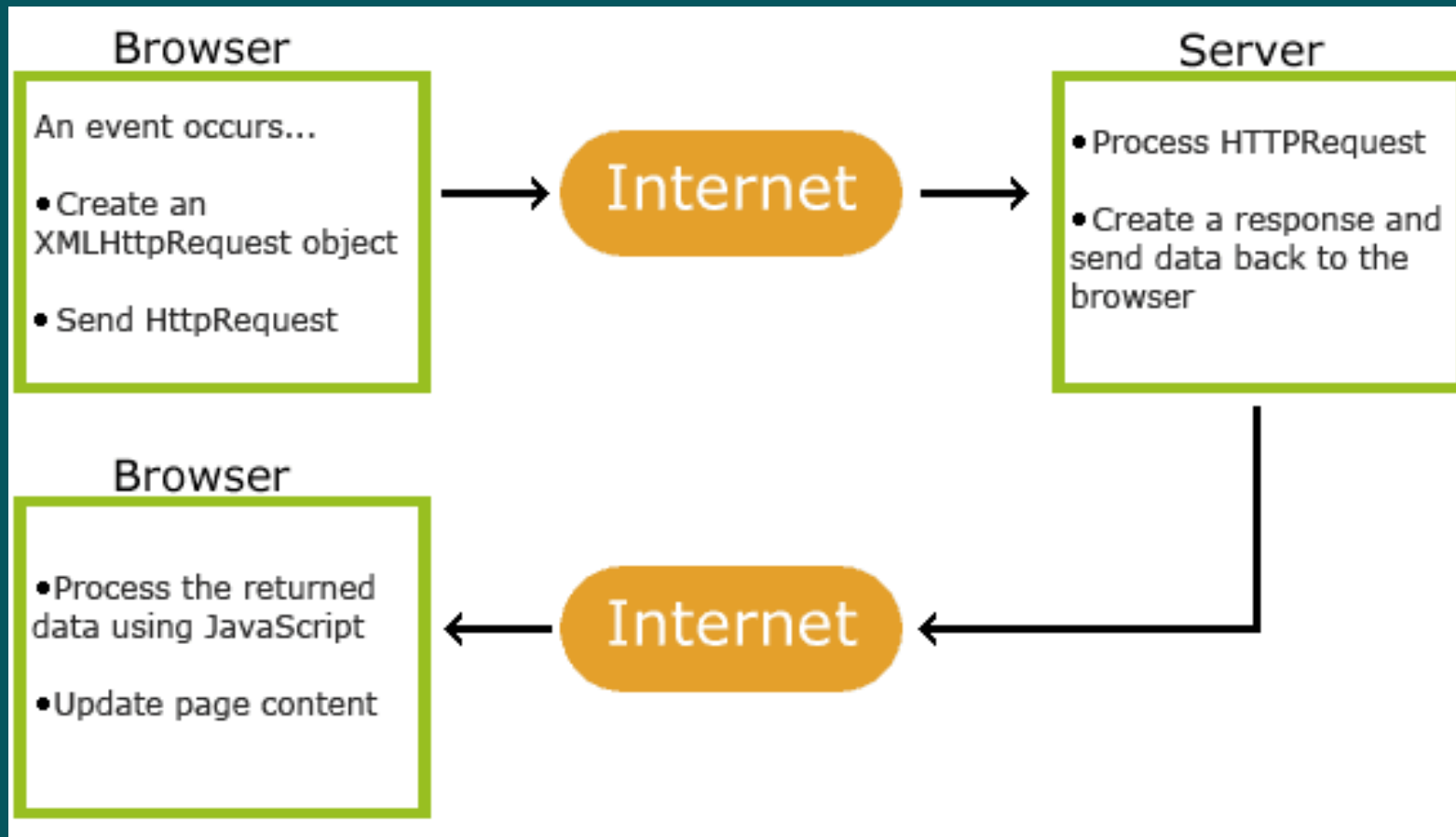
Asynchronous Call



Asynchronous Call



How AJAX Works



A synchronous

J avascript

A

X

JavaScript Objects

- Use the XMLHttpRequest (XHR) Object
 - Used to exchange data asynchronously with a server
 - It update parts of a web page, without reloading the whole page
 - It is the keystone of AJAX

XHR Object

Attributes

- readyState
- .responseText
- status
- statusText

Events

- onreadystatechange
- onload
- onloadstart
- onprogress

Methods

- open
- send
- abort

A synchronous

Javascript

A_{nd}

X_{ML}

Response of an AJAX Call

- XML
 - Used in earlier days
- JSON (JavaScript Object Notation)
 - Now the most popular data standard used
- HTML
 - Also used popularly
 - Mainly when the data sent from the server is for display as-is

AJAX Step-by-Step

STEP 1: Create XHR Object

```
var xhr = new XMLHttpRequest();
```

- **xhr** is the name of the object {any name can be used}
- **new** is the keyword used to create a **new Object**
- **XMLHttpRequest** is the name of the **object's class**

AJAX Step-by-Step

STEP 2: Create a Callback Function

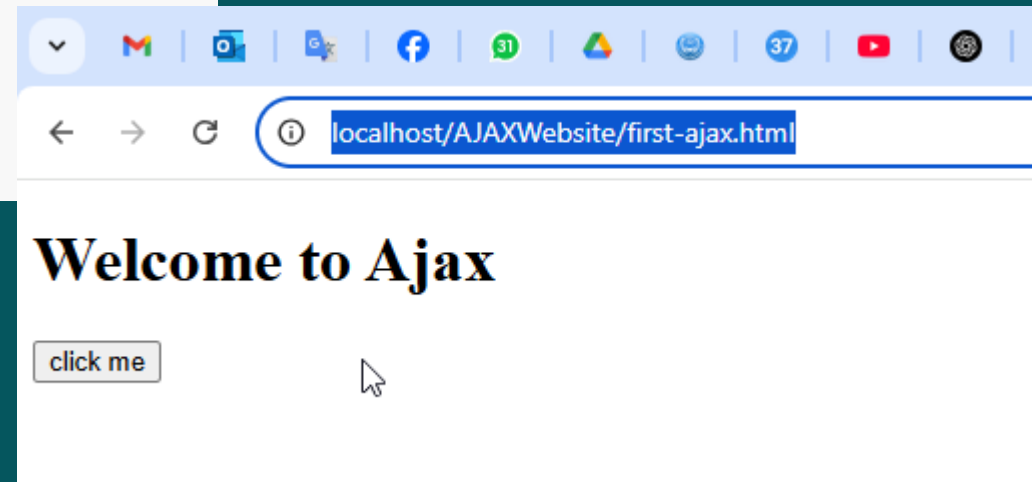
```
xhr.onreadystatechange = function () {  
  
  
  
  
}
```

- **onreadystatechange** event is fired every time there is a ***change*** in the event
- using anonymous function for call back (remember JavaScript class)

- Create new **folder** and make new **html** file in it
- Put **h1**, **button**, **div** in your new page
- Run it from *localhost* – after running your webserver(**xampp**)



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Document</title>
6 </head>
7 <body>
8     <h1>Welcome to Ajax</h1>
9     <button id="btn">click me</button>
10    <div id="demo"></div>
11 </body>
12 </html>
```



```
3 </head>
4   <meta charset="UTF-8">
5   <title>Document</title>
6 </head>
7 <body>
8   <h1>Welcome to Ajax</h1>
9   <button id="btn" onclick="actionMeth()">click
10  me</button>
11   <div id="demo"></div>
12 </body>
13 <script>
14   var xhr = new XMLHttpRequest();
15   console.log(xhr);
16   function actionMeth(){
17
```

The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the output of `console.log(xhr);` from the code on the left. The output is an `XMLHttpRequest` object, which is expanded to show its properties. The `onreadystatechange` property is highlighted by the mouse. The console also shows the file and line number `first-ajax.html:14` at the top right.

first-ajax.html:14

XMLHttpRequest 1

- `onabort`: null
- `onerror`: null
- `onload`: null
- `onloadend`: null
- `onloadstart`: null
- `onprogress`: null
- `onreadystatechange`: null
- `ontimeout`: null
- `readyState`: 0
- `response`: ""
- `responseText`: ""
- `responseType`: ""
- `responseURL`: ""
- `responseXML`: null
- `status`: 0
- `statusText`: ""
- `timeout`: 0
- `upload`: XMLHttpRequestUpload {onloadstart: null, onprogress: null, onreadystatechange: null, ontimeout: null, withCredentials: false}
- `[[Prototype]]`: XMLHttpRequest

Using xhr to open connection with server

```
}  
xhr.open((String method, String url, [bool async], [String user], [String password])  
'GET', 'getInfo.txt', true);
```



```
<script>
var xhr = new XMLHttpRequest();
    console.log(xhr.readyState);
```

0

```
<script>
var xhr = new XMLHttpRequest();
    xhr.open('GET', 'getInfo.txt', true);
    console.log(xhr.readyState);
```

1

```
<h1>Welcome to Ajax</h1>
<button id="btn" onclick="actionMeth()">click me</button>
<div id="demo"></div>
```

```
<script>
var xhr = new XMLHttpRequest();

function actionMeth(){
    console.log(xhr.readyState);
}
xhr.open('GET', 'getInfo.txt', true);
xhr.send();

</script>
```

4

Only after
press click me

readyState

This variable holds the status of the XMLHttpRequest. Changes from 0 to 4.

- 0 – Request not initialized
- 1 – Server connection established
- 2 – Request Received
- 3 – Processing Request
- 4 – Request Finished and response is ready

```
function actionMeth(){  
    // console.log(xhr.readyState);  
    xhr.onreadystatechange= function()  
    {  
        if(xhr.readyState==4 && xhr.status ==200)  
            document.getElementById("demo").innerHTML=this.responseText;  
    }  
    |  
    xhr.open('GET','getInfo.txt',true);  
    xhr.send();  
}
```

this = xhr

Welcome to Ajax

click me
welcome Sir

status

This is the same as the status of the HTTP response codes.

200 – OK

404 – Page not found

403 – "Forbidden"

.....

```
xhr.onreadystatechange= function()  
{  
  if(xhr.readyState==4 && xhr.status ==200)  
    console.log("found");  
  elseif(xhr.status ==404)  
    console.log("not found");  
}  
  
}  
xhr.open('GET','getInfo21.txt',true);  
xhr.send();
```

✖ GET <http://localhost/AJAXWebsite/getInfo21.txt> 404 [first-ajax.html:27](#) ⓘ💡
(Not Found)
(anonymous) @ [first-ajax.html:27](#)

AJAX Step-by-Step

STEP 3: Do something when the request completed and response is ready

```
xhr.onreadystatechange = function () {  
  
    if( xhr.readyState == 4 && xhr.status == 200)  
    {  
  
    }  
  
}
```

AJAX Step-by-Step

STEP 4: Send the Request to the Server

```
xhr.open("GET","process.php",true);  
xhr.send();
```

- The open method specifies the type of request, the URL, and if the request should be handled

asynchronously or not

- SYNTAX: open(method, url, async)

method: the type of request: GET or POST

url: the location of the file on the server

async: true (asynchronous) or false (synchronous)

- The send method sends the request off to the server

The final step in the request handler is to send the request to the server.

- Has parameters which are used only in POST request

Complete Code

```
var xhr = new XMLHttpRequest();

xhr.onreadystatechange = function () {

    if( xhr.readyState == 4 && xhr.status == 200)
    {
        console.log(xhr.responseText);
    }
}

xhr.open("GET","process.php",true);
xhr.send();
```


Using synchronous method

```
var xhr = new XMLHttpRequest();
console.log(xhr.readyState);
function actionMeth(){
    // console.log(xhr.readyState);
/*    xhr.onreadystatechange= function()
    {
        if(xhr.readyState==4 && xhr.status ==200)
            document.getElementById("demo").innerHTML=this.responseText;
    }
*/
    xhr.open('GET','getInfo.txt',false);
    xhr.send();
    document.getElementById("demo").innerHTML=xhr.responseText;
}
```

Welcome to Ajax

click me

welcome Sir Mohammed



Most commonly used method(onload)

```
var xhr = new XMLHttpRequest();  
console.log(xhr.readyState);  
function actionMeth(){  
  
    xhr.onload= function(){  
        if(this.status ==200)  
            document.getElementById("demo").innerHTML=this.responseText;  
        //recieve response  
  
    }  
    xhr.open('GET','getInfo.txt',true); // connect to server  
    xhr.send();// send request
```

Welcome to Ajax

click me

welcome Sir Mohammed



AJAX with JQuery

- AJAX with JQuery becomes much simpler
- No need for so many steps as with using XHR Object
- Only one function called \$.ajax()
- It includes everything such as the method, url, data and the callback function

\$.ajax() function in JQuery

```
$.ajax({  
    url: 'process.php',  
    type: 'POST',  
    crossDomain: true,  
    data: { 'key': value, ..... },  
}).done(function(data) {  
    if(data==="success"){  
  
        }else{  
  
        }  
}).fail( function() {  
    });
```

Conclusion

- Data from HTML can be sent to the Server using two basic method GET and POST
- Data can be sent using two different ways: synchronous and asynchronous
- AJAX is used for asynchronous data transfer